

File handling:

- Types of data used for input output
- (1) Text - '12345' as sequence of unicode characters
 - (2) Binary - 1234 as a sequence of bytes of its binary equivalent

→ Here are 2 file types to deal with:

- (1) Text files - All program files are text files
- (2) Binary files - Image, music, video, exe files

* How file I/O is done in most programming languages

- Open a file
- Read/write data
- Close the file.

(1) Writing to a file

(1) If file is not present

`f = open('file.txt', 'w')`

`f.write("Hello world")`

`f.write("\n Now are you?")`

`f.close()`

file name

open in write mode

multiline

Q) If file is already present:

```
f = open ('path of file', 'w')
```

```
f.write ('Salman Var')
```

```
f.close()
```

this will replace
old content of
file,

How open() works (present in documentation of open() function)

→ file is present in Hard drive

→ if you write the code for opening file, python loads the file into your RAM, loads file into buffer memory where file is read character by character.

→ Until you are performing operations on file, the file is present in buffer memory.

→ If you close the file the file is removed from buffer and saved to hard drive.

problem with w mode

→ It replaces old content with new one. so we use append mode.

→ To write to file without removing old content we use append mode.

```
f = open('filepath', 'a')  
f.write("text you wanna write")
```

append mode

write line function

→ If you want to write multiple lines in single line of code

```
f L = ['hi', 'Hello', 'How are you']  
f = open('file.txt', 'w')  
f.writelines(L)  
f.close
```

(2) Reading a file

(1) using read()

```
f = open('filename', 'r')  
s = f.read()  
print(s)  
f.close
```

r mode

(2) reading upto n character

```
f.read(10)
```

characters you want to read

③ reading line by line
f.readline()

→ If your file contain too much content you use readline() so that your memory has lesser load.

Context manager (with)

- ① It's good idea to close a file after usage as it will free up the resources
- ② If we don't close it, garbage collector would close it.
- ③ with keyword closes the file as soon as the usage is over.

with open('file.txt', 'r') as f:
f.read()

seek and tell function

→ ~~read~~ seek() function moves the buffer pointer to the location that you give it as parameter

→ tell() function ~~reads~~ tells where the

Problem with working in text mode

- 1) Can't work with binary files like images.
- 2) not good for other data types like int/float/list/tuples

→ so we use binary files.

① with open('img.png', 'r') as f
f.read()

will throw an error

→ for text file data always should be unique unicode character and it doesn't work with other data.

so we use

① with open('img.png', 'rb') as f
f.read()

'rb' → read binary mode

problem 2

with open('f.txt', 'w') as f:
f.write(5)

we are trying to write integer datatype in file so it won't support it

→ to remove this problem we use serialization/deserialization techniques.

→ If you want to write dictionary datatype in file you can't write it until you change it into string. After changing it you cannot perform the dictionary operation in it. And you cannot change that string file into dictionary.

→ It is the main problem.

So we use serialization / deserialization techniques.

serialization & Deserialization

① Serialization : It is process of converting python datatypes to JSON

② Deserialization : process of converting JSON to python datatypes.

→ with open('demo.json', 'w') as f:

json.dump(L, f)

to write to json file

The file representator

the data you want write

like list, dictionary, tuple

json.load(f) → to read the file

→ `json.dump` write tuple as list in the file

serializing & deserializing custom objects

→ say you have to serialize object of class you cannot dump it as regular

→ You have to first create a function that tells ~~how~~ in what format you want to dump as you cannot dump as it is

→ then you pass the argument as

`json.dump (obj-name, f, default = function-name)`

→ So using `json.dump` you convert your object into string or certain datatype then finally dump

→ If you want same object from the file you cannot get it as its format is changed.

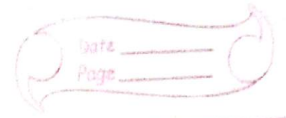
→ So we use concept of pickling.

pickling

→ pickling is process whereby a python obj hierarchy is converted into a byte-stream.

→ You can unpickling is inverse operation whereby a

byte stream is converted back to obj hierarchy.



So pickle is same as json, all functions are same
pickle.dump(), pickle.load.

→ The main difference is

- pickle lets the user to store data in binary format.
- JSON lets the user store data in a human-readable text format.